Pashov Audit Group

BNB Chain Security Review

April 30th 2026 - May 1st 2026

Contents

1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over $100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work here or reach out on Twitter @pashovkrum.

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

• **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
• **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
• **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

• **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
• **Medium** - only a conditionally incentivized attack vector, but still relatively likely
• **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. Methodology

Pashov Audit Group conducts each engagement as a team that works through the codebase end-to-end: we map the system, hunt for bugs, and then collaborate with the client to remediate the findings. This report tracks every commit reviewed, every finding raised, and the resolution status of each issue. Our auditors draw on experience from private engagements and public contests, and pick their own toolset per project — static analysis, fuzzers, formal verification, and AI-assisted review — based on what fits the codebase.

5. About BNB Chain

BNB Chain is a reference implementation of BEP-677, introducing EIP-8056 Scaled UI Amount support for BEP-20 tokens on BNB Smart Chain. It provides an updatable `uiAmountMultiplier` that wallets use to display scaled token balances without altering on-chain raw amounts, enabling stock-split-style redenominations and RWA adjustments at the display layer.

The token allows holders to request a UI multiplier change for a future timestamp. When that timestamp is reached, all wallet displays scale balances by the new multiplier without minting or burning tokens—analogous to a stock split at the display layer.

The system is composed of:

- `ERC8056BaseUpgradeable` — abstract base implementing the multiplier scheduling logic: `setUIMultiplier(newMult, timestamp)` queues a change, `uiMultiplier()` auto-advances to the pending value when `block.timestamp` passes the effective time, and `toUIAmount(rawAmount)` applies the multiplier via `rawAmount * multiplier / 1e18`
- `ERC8056TokenUpgradeable` — concrete BeaconProxy-deployable token that adds owner-only authorization via `_authorizeMultiplierUpdate()` and emits `TransferWithUIAmount(from, to, rawAmount, uiAmount)` on every transfer per BEP-677
- `ERC8056Base` / `ScaledUIToken` — non-upgradeable reference implementations retained for comparison; production deployments should use the upgradeable variant

Upgrade & deployment pattern

All token proxies delegate to a single `UpgradeableBeacon`; upgrading the Beacon upgrades all proxies atomically. The `initialize(initialOwner, name, symbol, initialSupply)` function has `initializer` modifier with no front-run protection—deployment scripts must bundle proxy creation and initialization atomically.

6. Executive Summary

A time-boxed security review of the **bnb-chain/bep-677-contracts** repository was done by Pashov Audit Group, during which **h2134, Hunter, Klaus, unforgiven** engaged to review **BNB Chain**. A total of **5** issues were uncovered.

Protocol Summary	
Project Name	BNB Chain
Protocol Type	EIP-8056 token implementation
Timeline	April 30th 2026 - May 1st 2026

Review commit hash:
- 13a604baa4d40f216bf6e561a32fb8e585440e96
  (bnb-chain/bep-677-contracts)

Fixes review commit hash:
- 36ab09954f7ffed52a08e4d5eb9d82aff694f057
  (bnb-chain/bep-677-contracts)

Scope

`ERC8056BaseUpgradeable.sol` `ERC8056TokenUpgradeable.sol` `IScaledUIAmount.sol` `IERC8056Scheduled.sol` `IScaledUIAmountBalances.sol` `IScaledUIAmountConversion.sol` `IScaledUIAmountNewUIMultiplier.sol`

7. Centralization & Trust

The protocol has 1 privileged role:

- **Owner** — instant `setUIMultiplier(newMultiplier, timestamp)` scheduling for any future timestamp with no minimum delay (can set t = block.timestamp + 1), override of pending multiplier changes without revert, `upgradeTo()` authorization on the UpgradeableBeacon (instant, all BeaconProxy instances upgrade atomically), and receives entire initial supply on `initialize()`

8. Findings

Findings count

Severity	Amount
Low	5
Total findings	**5**

Summary of findings

ID	Title	Severity	Status
[L-01]	Missing `UIMultiplierUpdated` event during initialization	Low	**Resolved**
[L-02]	Rounding down for all calculations without proper documentation	Low	**Resolved**
[L-03]	Pending multiplier overwrites can accelerate an already announced effective time	Low	**Resolved**
[L-04]	Missing input bounds validation in production ready contract	Low	**Resolved**
[L-05]	Adopt ERC-7201 namespaced storage for safer upgrade	Low	Acknowledged

Low findings

[L-01] Missing `UIMultiplierUpdated` event during initialization

Description

The EIP-8056 specification mandates that the `UIMultiplierUpdated` event MUST be emitted whenever the multiplier is changed. Within the `__erc8056Base_init_unchained` function, the contract initializes the internal `_uiMultiplier` state to 1e18 (representing the 1.0x baseline) but fails to log this transition via the required event. This oversight creates a "silent" state change that remains invisible to off-chain indexers, subgraphs, and analytics platforms. Without this initial event, third-party integrators lack a verifiable on-chain starting point for the token's scaling logic, which can lead to broken historical data or incorrect UI balance displays. To achieve full compliance, the contract should explicitly emit a `UIMultiplierUpdated` event during the initialization process to establish a transparent and traceable audit trail.

[L-02] Rounding down for all calculations without proper documentation

Description

The `toUIAmount` and `fromUIAmount` functions utilize standard integer division via `Math.mulDiv`, which inherently truncates results toward zero and leads to a loss of mathematical symmetry.

```
    function toUIAmount(uint256 rawAmount) public view virtual override returns (uint256) {
        return rawAmount.mulDiv(uiMultiplier(), MULTIPLIER_DECIMALS);
    }

    function fromUIAmount(uint256 uiAmount) public view virtual override returns (uint256) {
        return uiAmount.mulDiv(MULTIPLIER_DECIMALS, uiMultiplier());
    }
```

Because the EIP does not define a specific rounding strategy, this truncation means that a "round-trip" conversion (such as calling `fromUIAmount(toUIAmount(x))`) may result in a value lower than the original input, creating "dust" discrepancies that can confuse off-chain integrators or result in minor accounting errors for external protocols. While this is standard behavior for Solidity's fixed-point arithmetic, the lack of explicit documentation or rounding-direction controls (e.g., rounding up for specific UI-adjusted balances) could lead to unexpected behavior in front-ends or downstream DeFi integrations that expect perfectly reversible conversions.

[L-03] Pending multiplier overwrites can accelerate an already announced effective time

Description

The contract allows the owner to overwrite a pending multiplier with a new `effectiveAtTimestamp` that is earlier than the currently queued timestamp.

```
require(effectiveAtTimestamp > block.timestamp, "ERC8056: effective time must be in future");
if (block.timestamp < _nextUiMultiplierEffectiveAt
    && _nextUiMultiplierEffectiveAt != type(uint256).max) {
    _onMultiplierOverwrite(
        _nextUiMultiplier,
        _nextUiMultiplierEffectiveAt,
        newMultiplier,
        effectiveAtTimestamp
    );
}
_nextUiMultiplier = newMultiplier;
_nextUiMultiplierEffectiveAt = effectiveAtTimestamp;
```

The only timing check is that the new timestamp is in the future. There is no check that an overwrite preserves or extends the already announced effective time.

`_setUIMultiplier` in `ERC8056BaseUpgradeable.sol` (line 369) is the directly affected internal function where the timing check `require(effectiveAtTimestamp > block.timestamp)` exists, but no minimum lead time is enforced.

Example:

```
setUIMultiplier(2e18, block.timestamp + 30 days);
setUIMultiplier(3e18, block.timestamp + 1);
```

Integrators watching the first schedule may prepare around a 30-day notice window, but the owner can replace it with a multiplier that activates almost immediately. `UIMultiplierChangeOverwritten` gives observability, but it does not give integrators any guaranteed reaction time after the overwrite.

Recommendation

Either prevent pending schedule acceleration:

```
if (hasPendingMultiplier()) {
    require(
        effectiveAtTimestamp >= _nextUiMultiplierEffectiveAt,
        "cannot accelerate pending multiplier"
    );
}
```

or require every update, including overwrites, to satisfy a fresh minimum notice period:

```
require(effectiveAtTimestamp >= block.timestamp + MIN_NOTICE, "notice too short");
```

Alternatively, if enforcing a minimum lead time in code is undesirable, add an explicit warning to the BEP-677 SECURITY CONSIDERATIONS section documenting that the contract does not enforce any minimum notice period and that integrators must rely on governance controls or off-chain monitoring for protection.

[L-04] Missing input bounds validation in production ready contract

Description

The `ERC8056TokenUpgradeable` contract is advertised in the README as "Ready to deploy — no subclassing required." However, it fails to override the `_validateMultiplier` function, inheriting the base implementation, which only enforces `newMultiplier > 0`.

This lack of bounds allows the owner to set extreme multipliers, leading to critical UI failures:

- **Precision Loss (Too Small):** Setting the multiplier to 1 will cause `toUIAmount()` to round down to 0 for any raw balance smaller than $10^{decimals}. Small holders' `balanceOfUI()` will falsely return 0.
- **Frontend DoS (Too Large):** Setting a massive multiplier (e.g., 1e50) will cause `totalSupplyUI()` and `balanceOfUI()` to permanently revert due to uint256 overflow in Math.mulDiv, breaking block explorers and integrated wallets.

`_update()` is also an affected function: when an extreme multiplier is active, the `TransferWithUIAmount(..., toUIAmount(value))` call inside `_update()` overflows in `Math.mulDiv` and reverts the entire ERC20 transfer, breaking `transfer()` and `transferFrom()` for all token holders.

Recommendation:

Override `_validateMultiplier` directly in `ERC8056TokenUpgradeable` to enforce safe operational limits appropriate for the token's use case, rather than relying on developers to subclass it. Alternatively, wrap the `toUIAmount(value)` call inside `_update()` in a try/catch block so that an overflow in the UI amount calculation does not revert the underlying raw ERC20 transfer; in the catch branch, either skip the `TransferWithUIAmount` event or emit it with a zero UI value.

[L-05] Adopt ERC-7201 namespaced storage for safer upgrade

Description

Every parent in the inheritance chain — `ERC20Upgradeable`, `OwnableUpgradeable`, `Initializable`, `ContextUpgradeable` — is OpenZeppelin v5 and uses ERC-7201 namespaced storage. `ERC8056BaseUpgradeable` is the only contract in the chain that still carries a `__gap`.

Maintaining `__gap` arithmetic across upgrades is error-prone. A future v2 that adds one storage variable must simultaneously decrement the gap from 47 to 46 and place the new variable at slot 50; a mismatch on either side silently shifts every downstream slot and corrupts state for every BeaconProxy. ERC-7201 removes this class of mistake — fields go inside a struct anchored at a deterministic namespaced location, and new state never collides with any other slot. It is recommended to move the storage variables into a namespaced storage struct and delete `__gap`.

Client Commentary

The current implementation retains the existing `__gap` pattern, judging the ERC-7201 migration as over-engineering for a stable 3-slot contract. As mitigation, the NatDoc on `__gap` was rewritten from a soft description into a numbered maintainer checklist that requires new variables to be declared before `__gap`, the gap size to be decremented by the exact slot count consumed, no reordering of existing variables, an `npx hardhat test` run to invoke the OpenZeppelin `validateUpgrade` layout check on every upgrade, and a review of OpenZeppelin release notes before upgrading any parent contract. The upgrade safety test describe block was relabeled (L-05) to tie the automated layout guard explicitly to this finding, and a forward-looking note was added pointing to ERC-7201 should extensive storage additions become necessary in the future.